Elements of AIML
Unit 03 – Lecture 02 Notes
Train/Test/Validation Split & Cross-Validation

Tofik Ali

March 28, 2026

Contents

1 Lecture Overview **2**

2 How to use these notes (self-study) **2**

3 Learning Outcomes **2**

4 Before you start **2**

5 Keywords **2**

6 Abbreviations and symbols **3**

7 Why splits are necessary **3**
7.1 Generalization gap (quick intuition) . . . 3

8 Train/validation/test **3**
8.1 Train set . . . 3
8.2 Parameters vs hyperparameters . . . 3
8.3 Validation set . . . 3
8.4 Test set . . . 4
8.5 Typical split ratios (rule of thumb) . . . 4

9 k-fold cross-validation **4**
9.1 What CV is used for (two different goals) . . . 4
9.2 Stratified k-fold (classification) . . . 4
9.3 Other splitting variants (when needed) . . . 4

10 Data leakage (examples) **4**
10.1 More leakage patterns to watch for . . . 5
10.2 Correct preprocessing rule . . . 5

11 Worked scenario: churn prediction evaluation **5**

12 Minimal code patterns **5**
12.1 Train-test split . . . 5
12.2 Stratified k-fold . . . 6
12.3 Leakage-safe pipelines (recommended) . . . 6

13 Common pitfalls (and how to avoid them) **6**

14 Practice (with answers) **6**

15 Exit questions (with answers) **7**

16 Summary **7**

1 Lecture Overview

This lecture focuses on **evaluation methodology**: how to split data, how to tune models, and how to avoid common mistakes that produce misleading performance numbers.

Sources: Mueller and Massaron, *Machine Learning for Dummies*, For Dummies, 2016.;Mitchell, *Machine Learning*, selected chapters on learning basics and evaluation.;Scikit-learn documentation, accessed 2026-02-28.

2 How to use these notes (self-study)

Whenever you see an evaluation number (accuracy/F1/RMSE), ask: "Which data was this computed on, and was any information from test leaking into training?". This lecture gives you a repeatable workflow for honest evaluation that you should follow in labs and projects.

3 Learning Outcomes

- Explain train/validation/test splits and what decisions each supports.
- Describe k-fold cross-validation and stratified splitting.
- Identify data leakage and explain why it is dangerous.
- Use a basic scikit-learn split pattern correctly.

4 Before you start

Before studying this lecture, make sure you can already identify:

- what the **features** and **labels** are in a dataset (**Unit 03 Lecture 01**),
- which task type you are solving: regression, classification, or clustering,
- why evaluation must reflect the real goal rather than only an impressive number (**Unit 01 Lecture 01**).

This lecture is about protecting the evaluation process from accidental self-deception.

5 Keywords

- **Overfitting:** model fits noise in training data; poor generalization.
- **Train set:** used to fit model parameters.
- **Validation set:** used to tune hyperparameters / select model.

- **Test set:** used for final unbiased evaluation.
- **Cross-validation:** repeated train/validate splits to estimate performance.
- **Leakage:** information from outside training-time availability enters the model.

6 Abbreviations and symbols

- **CV**: cross-validation; k: number of folds in k-fold CV.
- **Train/Val/Test**: training, validation, and test datasets.
- **Metric**: a number summarizing performance (accuracy, F1, RMSE, etc.).
- $\hat{y}$: model predictions; y: true labels/targets.
- **Hyperparameter**: setting chosen by us (e.g., depth, C, λ), not directly learned as weights.

7 Why splits are necessary

If we evaluate on training data, we measure how well the model memorized the data, not how well it generalizes. Splits simulate the "future unseen" setting.

7.1 Generalization gap (quick intuition)

Often training error is lower than test error:

$$\text{Train error} < \text{Test error}$$

The difference is the **generalization gap**. Good evaluation tries to estimate test performance *before* we deploy the model.

8 Train/validation/test

8.1 Train set

Used to learn parameters (e.g., weights in regression). We can run the training algorithm many times on the train set.

8.2 Parameters vs hyperparameters

- **Parameters** are learned from training data (e.g., weights w in linear regression).
- **Hyperparameters** are chosen by us and control learning (e.g., number of neighbors k in kNN, regularization λ, tree depth).

We typically tune hyperparameters using validation (or CV), not using the test set.

8.3 Validation set

Used for **model selection** and **hyperparameter tuning**. Examples of tuning decisions:

- choose model type (tree vs SVM),
- choose hyperparameters (regularization strength, depth),
- choose preprocessing options.

8.4 Test set

The test set is used **once** for the final report. If you repeatedly look at test results while tuning, the test set becomes part of training decisions and the estimate becomes optimistic.

8.5 Typical split ratios (rule of thumb)

There is no single best ratio, but common choices are:

- 80/20 (train/test) for quick experiments,
- 60/20/20 or 70/15/15 (train/val/test) when you need an explicit validation set.

When data is small, k-fold CV is often better than holding out a large validation set.

9 k-fold cross-validation

Cross-validation uses multiple train/validation splits to reduce variance in the estimate. In k-fold CV:

- split data into k folds,
- train on $k-1$ folds and validate on the remaining fold,
- repeat k times and average.

9.1 What CV is used for (two different goals)

- **Model selection**: choose the best hyperparameters/model family.
- **Performance estimation**: estimate how well the chosen model will generalize.

In practice, we often do model selection with CV and then train one final model on the full training set.

9.2 Stratified k-fold (classification)

Stratification preserves class proportions across folds to avoid folds with too few positive examples.

9.3 Other splitting variants (when needed)

- **Group split / GroupKFold**: keep samples from the same person/customer in one split (avoids "same-user" leakage).
- **Time-series split**: training uses past, validation uses future (no shuffling).
- **Nested CV (concept)**: outer CV estimates generalization, inner CV tunes hyperparameters (reduces optimistic bias).

10 Data leakage (examples)

Leakage can happen in subtle ways:

- **Post-event features:** a column that is only known after the outcome.

- **Preprocessing leakage:** scaling/encoding computed on the full dataset.
- **Duplicate leakage:** same or near-identical examples in train and test.

10.1 More leakage patterns to watch for

- **Target leakage**: a feature is a direct proxy for the label (e.g., "total_payment_received" to predict "loan default").
- **Feature selection leakage**: selecting features using the whole dataset before CV (the validation folds influenced feature choice).
- **Preprocessing order mistake**: "fit scaler" on all data, then split.

10.2 Correct preprocessing rule

Fit preprocessing only on training data:

- compute mean/std for scaling on train,
- apply the same transformation to validation/test.

11 Worked scenario: churn prediction evaluation

Suppose a telecom company has 1,000 customer records and wants to predict whether a customer will churn in the next month.

- **Task type:** binary classification.
- **Why stratify?** only about 120 customers have churned, so random splitting can create folds with too few positive examples.
- **One sensible workflow:** 80% train and 20% test; then use 5-fold stratified CV inside the training set to compare models.
- **Illustrative fold scores (F1):** 0.71, 0.75, 0.73, 0.70, 0.76. The mean is about 0.73, so that is the model-selection signal.
- **Final step:** retrain the chosen pipeline on the full training split, then report the test score only once.

The important lesson is not the exact numbers; it is the decision order: *split first, tune on training/CV, test once at the end.*

12 Minimal code patterns

12.1 Train-test split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Use ‘stratify=y‘ for classification problems when class balance matters. For regression, omit ‘stratify‘ and use a split strategy consistent with the data source (for example, time-based splits for forecasting).

12.2 Stratified k-fold

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for train_idx, val_idx in cv.split(X, y):
    X_tr, X_val = X[train_idx], X[val_idx]
    y_tr, y_val = y[train_idx], y[val_idx]
```

12.3 Leakage-safe pipelines (recommended)

Use a pipeline so that preprocessing is learned only from training folds:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression())
])
```

13 Common pitfalls (and how to avoid them)

- **Tuning on the test set:** if you look at test repeatedly, it stops being unbiased.
- **Preprocessing before splitting:** scaling/encoding on full data leaks information into test folds.
- **Wrong split strategy:** time-series and grouped data need special splits (no random shuffle).
- **Feature selection outside CV:** choosing features using all data leaks information into validation.
- **Reporting only one run:** randomness can change results; use CV or repeat splits when appropriate.

14 Practice (with answers)

P1. If you tune hyperparameters using the test set, what happens?

Answer: the test set is no longer unbiased; you indirectly overfit to it, so the reported test score becomes overly optimistic.

P2. Give one example where time-series splitting is required.

Answer: predicting next week’s sales from past sales. Shuffling would mix future information into training.

P3. Why is GroupKFold useful?

Answer: it prevents samples from the same group/person appearing in both train and validation/test, avoiding identity-based leakage.

15 Exit questions (with answers)

Q1. Why should the test set be used only at the end?

Answer: because otherwise tuning decisions indirectly depend on test performance, causing optimistic bias; the test set is no longer an unbiased estimate.

Q2. What is cross-validation and when is it useful?

Answer: CV repeats training/validation on multiple folds to get a more stable estimate; useful when data is limited or we want robust model selection.

Q3. Give two leakage examples.

Answer: using a future field to predict the outcome; scaling using the full dataset before splitting; duplicates across train/test.

Q4. What is stratification?

Answer: ensuring each split/fold has approximately the same class distribution as the full dataset, important for classification metrics.

16 Summary

Correct evaluation requires clean separation between training and final testing. Cross-validation and careful leakage prevention are core skills for reliable ML.

Sources: Mueller and Massaron, *Machine Learning for Dummies*, For Dummies, 2016.;Mitchell, *Machine Learning*, selected chapters on learning basics and evaluation.;Scikit-learn documentation, accessed 2026-02-28.